**Shiv Nadar University Chennai**
**Rajiv Gandhi Salai (OMR), Kalavakkam-603110**
**CS3801 - Web Technologies Lab**

# Exercise-8
# Development of a Simple Shopping Application Using the MERN Stack

**Aim:** Develop a simple shopping application using MongoDB, Express.js, React.js, and Node.js (MERN Stack). The application must support user registration, login validation, product retrieval and display, and user profile management using RESTful API operations.

## 1. Database Design: Create the following two collections in MongoDB.

A. Products Collection: Store the details of the products available in the shopping application with the following fields:

1. productId – Unique identifier for each product.
2. productName – Name of the product.
3. productPrice – Price of the product.
4. category – Category of the product.
5. quantity – Available quantity of the product.
6. description – Description of the product.

B. Users Collection: Store the registered user details with the following fields:

1. username – Name of the user.
2. emailId – Email address of the user (must be unique).
3. age – Age of the user.
4. password – Password of the user.

Use the same Users collection for both registration and login. A separate Login collection is not required.

## 2. Front-End Development: Develop the following three web pages using React.js.

### Page 1: User Registration

- Provide input fields for username, email ID, age, and password.
- Send the registration details to the Express.js backend using the HTTP POST method.
- Validate the input details at the backend.
- Check whether the email ID is already registered in MongoDB.
- If the email ID is not registered, store the user's details in the Users collection.
- After successful registration, display a message containing the registered username.

    - **Example: Successfully registered! Welcome, Raghavendra.**
- If the email ID is already registered or the input details are invalid, display an appropriate error message.

**Page 2: User Login**

- Provide input fields for email ID and password.
- Send the login credentials to the backend using the HTTP POST method.
- Validate the login details by checking the Users collection in MongoDB.
- If the email ID is not available in the database, return and display the message **"User Not Registered."**
- If the email ID exists but the password does not match, display "**Invalid Password."**
- If both the email ID and password match the stored details, allow the user to access the Products Display page.

**Page 3: Products Display**

- After successful login, retrieve the product details from the backend using "only the HTTP GET method".
- Fetch all products from the Products collection in MongoDB.
- Display the following details for each product:
    - Product ID
    - Product name
    - Product price
    - Category
    - Available quantity
    - Description
- Display an appropriate message if no products are available.
- Ensure that unauthenticated users cannot access the product data directly through the backend API.

**3. User Profile Management Using REST API**

Allow registered users to update or delete their registration details by implementing the following REST API operations.

A. PUT Operation — Update Complete Details

Update the user's complete set of editable profile details by passing the email ID as a query parameter and the updated details in the request body.

**Example request: PUT /api/users?emailId=user@example.com**

B. PATCH Operation — Update Selected Details

Update only the specified user fields by passing the email ID as a query parameter and the fields to be updated in the request body.

**Example request: PATCH /api/users?emailId=user@example.com**

C. DELETE Operation — Delete User Details

Delete the user's registration record by passing the email ID as a query parameter.

**Example request: DELETE /api/users?emailId=user@example.com**

The backend must check whether the specified user exists before performing the operation and return an appropriate success or error response.

## 4. Implementation Requirements

1. Create the MongoDB database and the Products and Users collections.

2. Develop the backend using Node.js and Express.js.

3. Connect the Express.js application to MongoDB using Mongoose.

4. Implement REST API endpoints for user registration, login validation, product retrieval, and user profile management.

5. Develop the three front-end pages using React.js functional components.

6. Use React hooks such as `useState()` and `useEffect()` wherever appropriate.

7. Use HTTP POST for registration and login, GET for product retrieval, PUT for complete profile updates, PATCH for partial profile updates, and DELETE for user deletion.

8. Display appropriate success and error messages based on the backend responses.

9. Test all operations using the React.js application and an API testing tool such as Postman or Thunder Client.

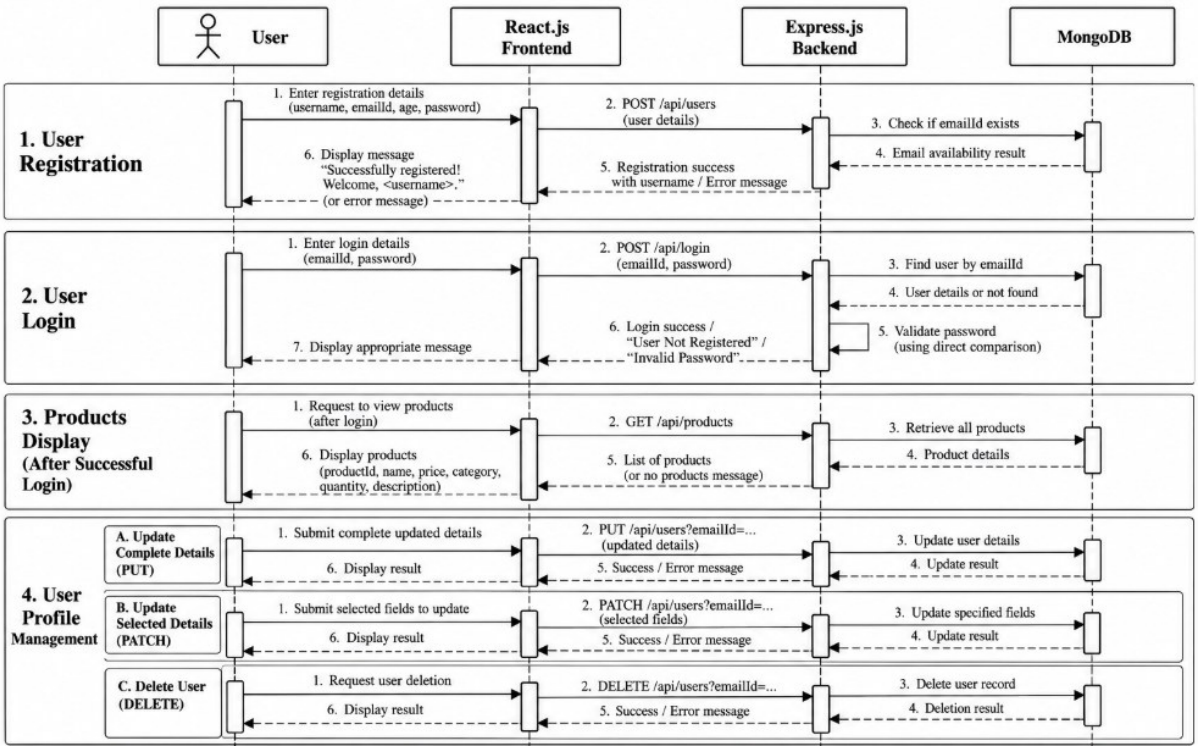
## 5. Expected Outcome

The developed application should enable users to:

- Register and receive a personalized registration success message.
- Log in by validating their email ID and password against MongoDB.
- Receive appropriate error messages for unregistered users and incorrect passwords.
- View product details retrieved using the HTTP GET method after successful login.
- Update their complete profile or selected profile fields using PUT and PATCH.
- Delete their registration details using DELETE.

## 6. Sequence Diagram

The following sequence diagram illustrates the interaction among the user, React.js frontend, Express.js backend, and MongoDB database.




## 7. Required API Endpoints

| HTTP Method | Endpoint | Purpose |
|---|---|---|
| POST | /api/users | Register a new user |
| POST | /api/login | Validate user login |
| GET | /api/products | Retrieve and display products |
| PUT | /api/users?emailId=user@email.com | Update complete editable profile details |
| PATCH | /api/users?emailId= user@email.com | Update selected profile fields |
| DELETE | /api/users?emailId= user@email.com | Delete a user's registration record |